

# Programmmentwurf Schachspiel

Name: Aburass, Anisa

Matrikelnummer: 8495063

Abgabedatum: Dienstag 10.06.2025

### Allgemeine Anmerkungen:

- *es darf nicht auf andere Kapitel als Leistungsnachweis verwiesen werden (z.B. in der Form “XY wurde schon in Kapitel 2 behandelt, daher hier keine Ausführung”)*
- *alles muss in UTF-8 codiert sein (Text und Code)*
- *sollten mündliche Aussagen den schriftlichen Aufgaben widersprechen, gelten die schriftlichen Aufgaben (ggf. an Anpassung der schriftlichen Aufgaben erinnern!)*
- *alles muss ins Repository (Code, Ausarbeitung und alles was damit zusammenhängt)*
- *die Beispiele sollten wenn möglich vom aktuellen Stand genommen werden*
  - *finden sich dort keine entsprechenden Beispiele, dürfen auch ältere Commits unter Verweis auf den Commit verwendet werden*
  - *Ausnahme: beim Kapitel “Refactoring” darf von vorne herein aus allen Ständen frei gewählt werden (mit Verweis auf den entsprechenden Commit)*
- *falls verlangte Negativ-Beispiele nicht vorhanden sind, müssen entsprechend mehr Positiv-Beispiele gebracht werden*
  - *Achtung: werden im Code entsprechende Negativ-Beispiele gefunden, gibt es keine Punkte für die zusätzlichen Positiv-Beispiele*
  - *Beispiele*
    - *“Nennen Sie jeweils eine Klasse, die das SRP einhält bzw. verletzt.”*
      - *Antwort: Es gibt keine Klasse, die SRP verletzt, daher hier 2 Klassen, die SRP einhalten: [Klasse 1], [Klasse 2]*
      - *Bewertung: falls im Code tatsächlich keine Klasse das SRP verletzt: volle Punktzahl ODER falls im Code mind. eine Klasse SRP verletzt: halbe Punktzahl*
- *verlangte Positiv-Beispiele müssen gebracht werden*
- *Code-Beispiel = Code in das Dokument kopieren*

## Kapitel 1: Einführung

### Übersicht über die Applikation

*[Was macht die Applikation? Wie funktioniert sie? Welches Problem löst sie/welchen Zweck hat sie?]*

Es kann Schach in der Kommandozeile gespielt werden. Vor jedem Zug wird das aktuelle Board (Spielbrett) ausgegeben, darauf muss der aktive Spieler die Stand-/Endposition seines Zuges angeben. Sofern das ein legaler Zug ist, wird dieser ausgeführt und der andere Spieler darf seinen Zug eingeben.

So lange bis das Spiel in einem Schachmatt oder Patt endet. Dabei kontrolliert das Programm jeden Zug auf seine Gültigkeit (darf die Figur sich so bewegen? steht der König im Schach? ist eine Figur im Weg? Usw..) und am Ende jedes Zuges wird das Board auf ein Matt oder Patt geprüft. Das Spiel wird automatisch beendet wenn das der Fall ist.

Die Gewinne der verschiedenen Spieler können gespeichert und als Leaderboard abgefragt werden (Beim GameInitializer 2 eingeben).

Zum Ende jedes Spiels wird der Spielverlauf in einem GameRecord gespeichert, diese Files befinden sich im Ordner "game\_records".

Ein laufendes Spiel kann unterbrochen werden, indem bei der Abfrage nach der Startposition "speichern" eingegeben wird. Das Spiel wird als GameState gespeichert und kann zu einem beliebigen späteren Zeitpunkt weitergespielt werden.

Es werden die Namen und die Farben der Spieler, der Spieler der aktuell am Zug ist und die Brettaufstellung gespeichert. Die GameState Files werden im Ordner "game\_state\_" gespeichert.

Um einen Spielstand weiterzuspielen muss beim Start (GameInitializer) eine 3 eingegeben werden. Dann werden die Dateinamen aller gespeicherten Spielstände angezeigt und es muss ein Spielstand ausgewählt werden. Der ausgewählte Spielstand wird geladen.

## Wie startet man die Applikation?

*[Wie startet man die Applikation? Welche Voraussetzungen werden benötigt? Schritt-für-Schritt-Anleitung]*

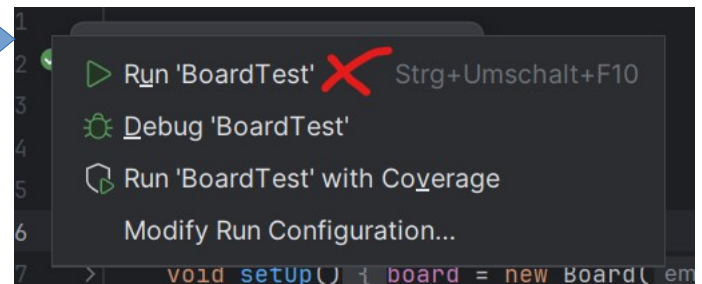
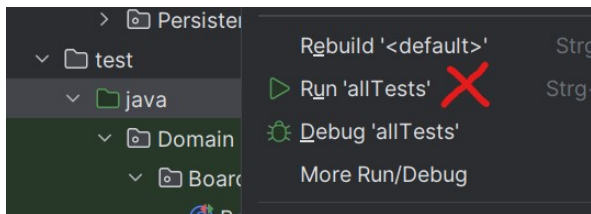
Es gibt zwei Möglichkeiten: Über die Main-Methode in den Klassen GameInitializer oder Quickstart im Package Application.

- GameInitializer: Auswählen zwischen „normalem Spiel“, „Leaderboard anzeigen“ und „angefangenen Spielstand laden“. Beim „normalen Spiel“ müssen erst die Namen der beiden Spieler eingegeben werden und dann wird das Spiel gestartet.  
Wenn „Leaderboard anzeigen“ ausgewählt wird, wird das lokale Leaderboard ausgegeben. Also die Spieler(namen), die an diesem Computer schon mal gespielt (und gewonnen) haben.  
Wenn ein Spielstand geladen werden soll, wird die Liste der Namen der gespeicherten Spielstände ausgegeben. Von diesen muss einer ausgewählt werden.
- Quickstart: Es wird ein „normales Spiel“ begonnen, die Spieler heißen A (weiß) und B (schwarz). Dabei muss nichts mehr eingestellt werden, das Spiel geht direkt los.

## Wie testet man die Applikation?

*[Wie testet man die Applikation? Welche Voraussetzungen werden benötigt? Schritt-für-Schritt-Anleitung]*

Über die IDE (ich hab IntelliJ benutzt) mit einem Rechtsklick auf das Package (src.test).java und dort „Run all Test“ auswählen. Alternativ kann in den jeweiligen Testklassen entweder oben an der Klassen Definition oder bei jedem Test selber am Rand auf das Startsymbol geklickt werden und den/die Tests auszuführen



## Kapitel 2: Clean Architecture

### Was ist Clean Architecture?

*[allgemeine Beschreibung der Clean Architecture in eigenen Worten]*

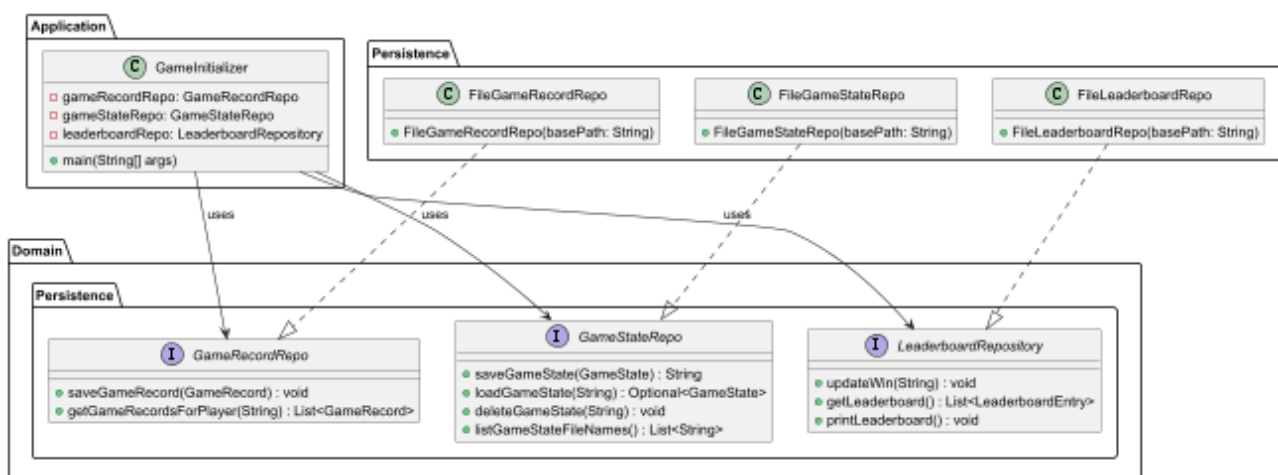
Clean Architecture ist ein Architekturstil, bei dem der Code in klar getrennte Schichten unterteilt wird. Dabei wird die Geschäftslogik (Domain) von technischen Details wie Datenbankzugriff, Frameworks oder Benutzeroberflächen isoliert. Dadurch wird das System modular, leichter test- und erweiterbar.

### Analyse der Dependency Rule

*[(1 Klasse, die die Dependency Rule einhält und eine Klasse, die die Dependency Rule verletzt); jeweils UML der Klasse und Analyse der Abhängigkeiten in beide Richtungen (d.h., von wem hängt die Klasse ab und wer hängt von der Klasse ab) in Bezug auf die Dependency Rule]*

### Positiv-Beispiel: GameInitializer.java und Repositories zum speichern

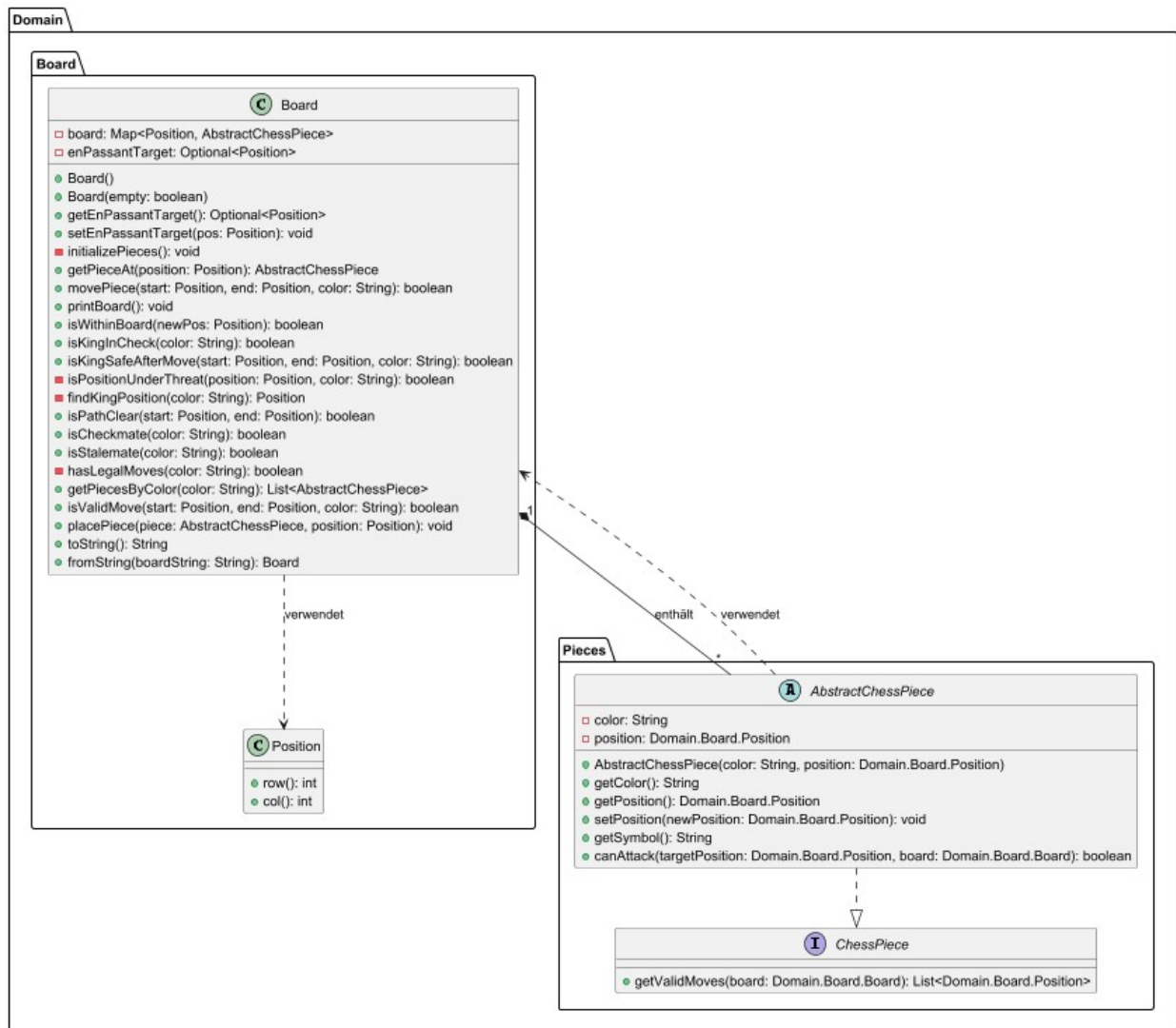
Die GameInitializer Klasse hängt von den Domain.Persistence Interfaces ab. Sie arbeitet ausschließlich mit Abstraktionen und ist nicht direkt an konkrete Klassen gekoppelt. Die konkreten Implementierungen (FileXYRepo) implementieren die Schnittstellen in einem eigenen Paket.



## Negativ-Beispiel: Board.java und AbstractChessPiece.java

Hier gibt es eine bidirektionale Abhängigkeit. Das Board benötigt die Figuren, um den Spielstand zu verwalten und Züge zu validieren und hängt von AbstractChessPiece ab.

Das AbstractChessPiece hat die Methode `canAttack(Position targetPosition, Board board)`. Bei dieser wird das Board als Parameter übergeben, sodass die Figuren (und damit deren abstrakte Oberklasse) auf den Zustand des Boards zugreifen müssen.



## Analyse der Schichten

*[jeweils 1 Klasse zu 2 unterschiedlichen Schichten der Clean-Architecture: jeweils UML der Klasse (ggf. auch zusammenspielenden Klassen), Beschreibung der Aufgabe, Einordnung mit Begründung in die Clean-Architecture]*

### **Schicht: Domain | Klasse: Board (UML siehe S. 5)**

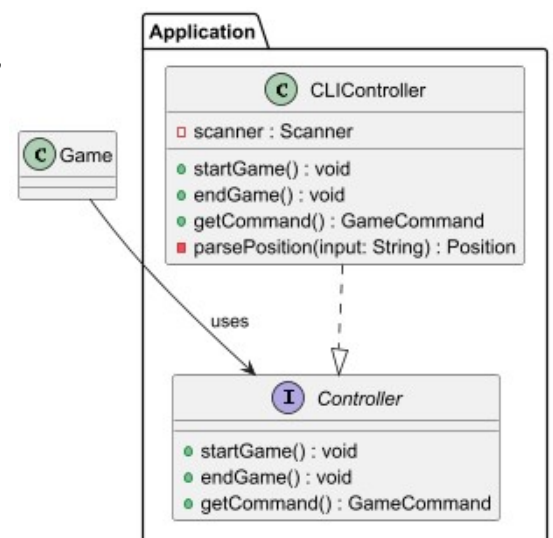
Die Klasse **Board** kapselt den gesamten Spielzustand und die Geschäftslogik des Schachspiels. Sie verwaltet die Aufstellung der Figuren, die Validierung von Zügen (z. B. ob der König im Schach steht) und andere zentrale Regeln des Spiels.

Das Board ist ein Kernbestandteil des Domain Models, da es die Regeln und den Zustand des Schachspiels abbildet. Es enthält keine technischen Details wie Benutzerinteraktion oder Persistenzmechanismen. So bleibt die Geschäftslogik rein und unabhängig von äußeren Schichten – ein zentrales Ziel der Clean Architecture.

### **Schicht: Application**

Der **CLIController** ist für die Interaktion mit dem Benutzer über die Kommandozeile verantwortlich. Er nimmt Eingaben entgegen, interpretiert sie (z. B. als Zugbefehl oder als Sonderbefehl wie „speichern“) und leitet sie an die Domänenschicht weiter, ohne selbst Geschäftslogik zu implementieren.

Der Controller ist Teil der Anwendungsschicht, weil er die Brücke zwischen der Benutzeroberfläche (oder den Eingaben) und dem Domain Model bildet. Er enthält keine Geschäftslogik, sondern dient lediglich dazu, Eingaben zu sammeln, zu validieren und in entsprechende Commands (z. B. MoveCommand oder SaveCommand) zu übersetzen. So bleibt die Domänenschicht sauber, während die Anwendungsschicht den konkreten Umgang mit externen Ein- und Ausgaben übernimmt



## Kapitel 3: SOLID

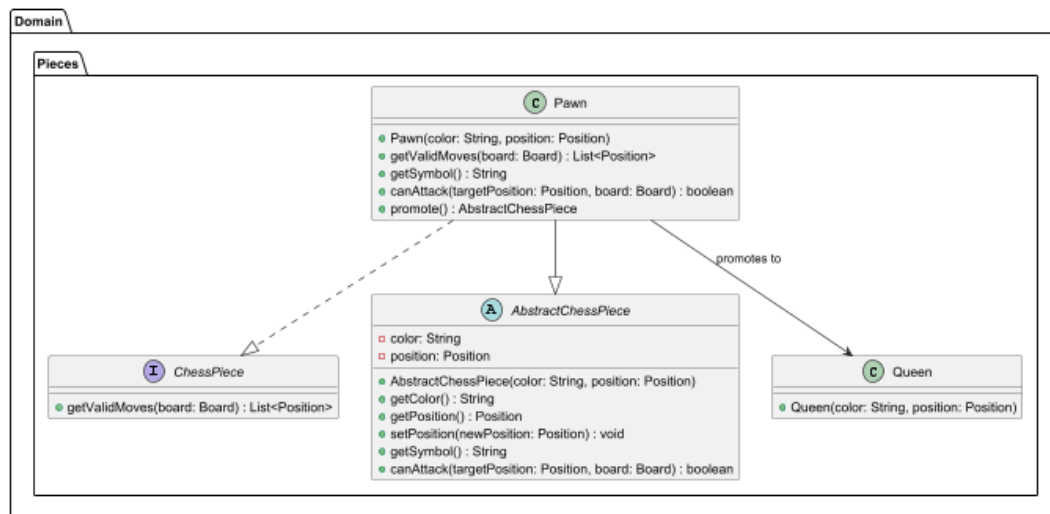
### **Analyse Single-Responsibility-Principle (SRP)**

*[jeweils eine Klasse als positives und negatives Beispiel für SRP; jeweils UML der Klasse und Beschreibung der Aufgabe bzw. der Aufgaben und möglicher Lösungsweg des Negativ-Beispiels (inkl. UML)]*

## Positiv-Beispiel:

Die Klasse **Pawn** (sowie die anderen Figuren) ist dafür zuständig, ihre eigenen Zugregeln zu implementieren (z. B. `getValidMoves`, `canAttack`) – einschließlich Sonderzügen wie En Passant.

Die Klasse ist fokussiert auf die Bewegungslogik des Bauern, ohne sich um übergreifende Zustandsverwaltung, Persistenz oder UI zu kümmern.



## Negativ-Beispiel:

fromString in Board:

```
public static Board fromString(String boardString) {
    Board board = new Board(true);
    String[] lines = boardString.split("\n");
    for (String line : lines) {
        if (line.trim().isEmpty()) continue;
        String[] parts = line.split("\\|");
        if (parts.length < 3) continue;
        Position pos = Position.fromString(parts[0].trim());
        String pieceType = parts[1].trim();
        String color = parts[2].trim();
        AbstractChessPiece piece = null;
        switch (pieceType.toLowerCase()) {
            case "pawn":
                piece = new Pawn(color, pos);
                break;
            case "knight":
                piece = new Knight(color, pos);
                break;
            case "bishop":
                piece = new Bishop(color, pos);
                break;
            case "rook":
                piece = new Rook(color, pos);
                break;
            case "queen":
                piece = new Queen(color, pos);
                break;
            case "king":
                piece = new King(color, pos);
                break;
        }
    }
}
```

```

        break;
    default:
        continue;
    }
    board.placePiece(piece, pos);
}

```

Die Methode übernimmt in dieser Form mehrere Aufgaben:

#### **Parsing der Eingabe:**

Die Methode liest den String ein, teilt ihn in Zeilen auf und verarbeitet diese, um Informationen wie Position, Figurentyp und Farbe zu extrahieren.

#### **Erzeugung der Schachfiguren:**

Außerdem entscheidet die Methode anhand eines switch-Statements, welche konkrete Schachfigur (z. B. Pawn, Knight etc.) erstellt werden soll. Dabei wird nicht nur die Logik zur Erzeugung von Objekten durchgeführt, sondern auch fest codiert, wie die Instanziierung je nach Typ erfolgt.

Wenn sich das Format der Eingabe (also der String) ändert, müsste die Methode angepasst werden, um den neuen Formatierungen gerecht zu werden. Änderungen an der Art, wie Schachfiguren instanziiert werden (z. B. zusätzliche Parameter, neue Figuren), können weitere Anpassungen in der Methode erfordern.

### **Analyse Open-Closed-Principle (OCP)**

*[jeweils eine Klasse als positives und negatives Beispiel für OCP; jeweils UML der Klasse und Analyse mit Begründung, warum das OCP erfüllt/nicht erfüllt wurde – falls erfüllt: warum hier sinnvoll/welches Problem gab es? Falls nicht erfüllt: wie könnte man es lösen (inkl. UML)?]*

#### **Zwei Positiv-Beispiele, 1.:**

In der Domänenschicht werden Repository-Interfaces (z. B. **GameRecordRepo** (UML siehe S. 4)) definiert, die die Persistenzoperationen (Speichern/Abrufen von Spielaufzeichnungen) beschreiben. Das Interface liegt hierbei in der Domain Schicht und die Implementierung in der Persistence Schicht.

Die Domain-Klassen hängen nur von abstrakten Repository-Interfaces ab. Möchte man z. B. den Speichermodus ändern (z. B. von Datei auf Datenbank), so kann man eine neue Implementierung schreiben, ohne die Domain-Klasse zu modifizieren.

#### **2.:**

Die Methode **getValidMoves()** in den Figuren (z. B. Pawn, Knight, Bishop) ist so implementiert, dass sie neue Sonderregeln (wie En Passant oder Doppelzug bei Bauern) durch Erweiterung der Methode einfügen können, ohne dass bestehender Code geändert werden muss. Damit bleibt der bestehende Code offen für Erweiterungen, ohne dass er modifiziert werden muss.



## Analyse Liskov-Substitution- (LSP), Interface-Segregation- (ISP), Dependency-Inversion-Principle (DIP)

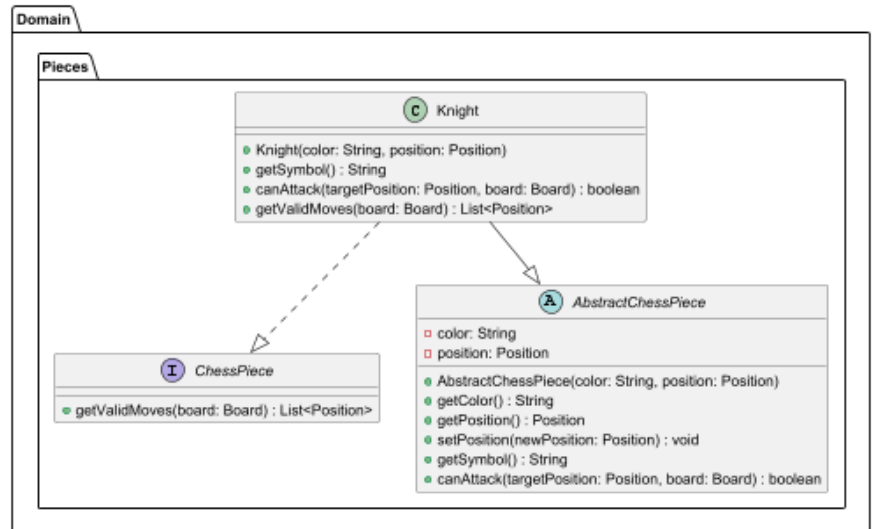
[jeweils eine Klasse als positives und negatives Beispiel für entweder LSP oder ISP oder DIP); jeweils UML der Klasse und Begründung, warum man hier das Prinzip erfüllt/nicht erfüllt wird]

[Anm.: es darf nur ein Prinzip ausgewählt werden; es darf NICHT z.B. ein positives Beispiel für LSP und ein negatives Beispiel für ISP genommen werden]

### Zwei Positiv-Beispiele für LSP | 1.:

Ein **Knight** (Springer) verhält sich so, wie es von der abstrakten Klasse **AbstractChessPiece** erwartet wird. Er liefert gültige Züge über seine `getValidMoves()` Methode und kann in Algorithmen (z. B. in der Brettvalidierung) eingesetzt werden.

Alle Figuren, die von **AbstractChessPiece** erben, erfüllen die vertraglichen Erwartungen, sodass ein Objekt vom Typ **Knight** überall dort verwendet werden kann, wo ein **AbstractChessPiece** erwartet wird.



### 2.:

Die abstrakte Klasse **Player** (UML siehe S. 17 bei Entity) definiert grundlegende Eigenschaften wie Name und Farbe. **HumanPlayer** erbt von **Player** und liefert eine konkrete Implementierung für einen menschlichen Spieler.

Sowohl **HumanPlayer** als auch (wenn es implementiert wäre) **ComputerPlayer** müssten den gleichen Vertrag erfüllen, nämlich dass sie als Spieler eindeutig identifizierbar sind (über `getName()` und `getColor()`).

Durch die Vererbung von **Player** kann jede konkrete Spielerimplementierung (z. B. **HumanPlayer**) überall dort eingesetzt werden, wo ein **Player** erwartet wird z. B. im **Game**, das beide Spieler als Parameter erhält.

Wenn das **Game** beim Wechseln des aktiven Spielers **Player**-Objekte verwendet, dann wird ein **HumanPlayer** nahtlos anstelle eines anderen **Player**-Typs (z. B. einem hypothetischen **ComputerPlayer**) eingesetzt. So wird die Spiellogik nicht durch unterschiedliche Implementierungen beeinflusst.

# Kapitel 4: Weitere Prinzipien

## Analyse GRASP: Geringe Kopplung

*[jeweils eine bis jetzt noch nicht behandelte Klasse als positives und negatives Beispiel geringer Kopplung; jeweils UML Diagramm mit zusammenspielenden Klassen, Aufgabenbeschreibung und Begründung für die Umsetzung der geringen Kopplung bzw. Beschreibung, wie die Kopplung aufgelöst werden kann]*

### Positiv-Beispiel

Die Klasse **Game** (siehe UML S.18 bei Aggregates) steuert den Spielablauf. Sie verwaltet den Zustand des Spiels (Board, MoveHistory, Spieler) und führt alle Geschäftslogiken (wie Zugvalidierung, Zustandswechsel und Spielende-Erkennung) durch. Dabei erhält sie ihre Abhängigkeiten (z. B. Repository-Interfaces, Controller) über den Konstruktor.

**Game** hängt ausschließlich von abstrakten Schnittstellen ab (z. B. GameRecordRepo, GameStateRepo, LeaderboardRepository) und Value Objects (Board, MoveHistory). Alle Abhängigkeiten werden per Constructor Injection übergeben, sodass **Game** nicht von konkreten Implementierungen abhängig ist.

**Game** konzentriert sich rein auf die Geschäftslogik und den Spielzustand. Technische Details wie Dateizugriffe (Persistenz) oder Benutzerinteraktion (Controller) werden über Abstraktionen eingebunden. Somit kann sich **Game** ändern, ohne dass sich die konkrete Implementierung der äußeren Schichten ändern muss.

### Negativ-Beispiel

Der **CLIController** (UML siehe S. 6 Schicht: Application) ist verantwortlich für die Benutzerinteraktion über die Kommandozeile. Er nimmt Eingaben (z. B. Positionen oder Sonderbefehle wie "speichern") direkt über System.in entgegen und gibt Ergebnisse direkt aus.

### Hohe Kopplung:

Der CLIController ist direkt an konkrete Systemressourcen (System.in, System.out) und an die konkrete Implementierung des Scanner gebunden.

Dadurch ist er schwer testbar (man kann z. B. keine einfache Unit-Test-Ersetzung für die Benutzereingaben vornehmen) und Änderungen an der Benutzerschnittstelle (z. B. Wechsel zu einer grafischen Oberfläche) erfordern umfassende Anpassungen in dieser Klasse.

### Lösungsansatz:

Um die Kopplung zu reduzieren, könnte man eine abstrakte Schnittstelle (z. B. InputOutputAdapter oder UserInterface) einführen, die nur die benötigten Methoden zur Eingabe und Ausgabe definiert.

Der CLIController arbeitet dann ausschließlich mit dieser Abstraktion, sodass die konkrete Implementierung (CLI, GUI, Web) austauschbar wird und die Klasse dadurch weniger direkt an Systemressourcen gekoppelt ist.

## Analyse GRASP: Hohe Kohäsion

*[eine Klasse als positives Beispiel hoher Kohäsion; UML Diagramm und Begründung, warum die Kohäsion hoch ist]*

Die Klasse **Position** (UML siehe S. 17 bei Value Object) repräsentiert ein Feld auf dem Brett als Zahlenpaar (row und col). Sie stellt sicher, dass diese Werte innerhalb des gültigen Bereichs (0 bis 7) liegen und bietet Methoden, um diese Koordinate in die übliche Schachnotation (z. B. "e2") umzuwandeln und aus dieser zurückzulesen

- Alle Felder und Methoden der Klasse Position sind ausschließlich dafür da, eine Brettcoordinate darzustellen und zu verarbeiten. Es gibt keine zusätzlichen Verantwortlichkeiten oder Nebensächlichkeiten in dieser Klasse.
- Als Record ist Position unveränderlich und repräsentiert einen festen, atomaren Wert.
- Alle Operationen (z. B. die Validierung im Konstruktor, die Konvertierung in Schachnotation in toString() und das Parsen in fromString()) sind direkt auf das Konzept einer Brettcoordinate ausgerichtet.

## Don't Repeat Yourself (DRY)

*[ein Commit angeben, bei dem duplizierter Code/duplizierte Logik aufgelöst wurde; Code-Beispiele (vorher/nachher); begründen und Auswirkung beschreiben]*

**Commit: 06f1244** In den isValidMoves() Methoden der Figuren wurde die Prüfung ob die Position erstellt werden darf umgestellt. Erst stand die konkrete Prüfung in den Klassen und dann wurde auf die bereits bestehende statische Hilfsmethode von der Positions Klasse umgestellt. So muss der Code zur Prüfung nicht in allen Figurenklassen stehen.

```
if (row < 0 || row > 7 || col < 0 || col > 7) {  
    break;  
}
```

wurde verändert zu:

```
if (!Position.isWithinBounds(row, col)) {  
    break;  
}
```

Methode in Position:

```
public static boolean isWithinBounds(int row, int col) {  
    return row >= 0 && row <= 7 && col >= 0 && col <= 7;  
}
```

# Kapitel 5: Unit Tests

## 10 Unit Tests

*[Nennung von 10 Unit-Tests und Beschreibung, was getestet wird]*

| Unit Test                                             | Beschreibung                                                                                                                                                                          |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BoardTest#isKingInCheck                               | Erkennt das Programm das der weiße König er im Schach steht und nachdem die Figur wegbewegt wurde die ihn bedroht, dass er nicht mehr im Schach steht.                                |
| KingTest#testMoveIntoThreatenedField                  | Der König sollte nicht in Felder laufen können auf denen er bedroht wird. (Nicht selber ins Schach laufen)                                                                            |
| KnightTest#testIgnorePieces                           | Der Springer sollte sich über andere Figuren bewegen können.                                                                                                                          |
| BishopTest#testBlockedByOwnPiece                      | Der Läufer sollte nicht durch eigene Figuren laufen können und auch nicht auf ein Feld ziehen auf dem eine eigene Figur steht.                                                        |
| PawnTest#testEnPassant                                | Wenn die Möglichkeit auf En Passant schlagen besteht, sollte das ein gültiger Zug sein. (Endfeld in validMoves Liste enthalten)                                                       |
| QueenTest#testKingSafety                              | Wenn der König im Schach steht, sollte die Dame keinen Zug machen der den König im Schach lässt und sie sollte (wenn der Zug legal) die Figur die den König bedroht schlagen können.  |
| PawnTest#testPawnPromotion White (und für Black)      | Wenn ein Bauer am Ende der Bretts angelangt ist (beim Gegner) wird er automatisch zur Dame promoviert.                                                                                |
| RookTest#testNoDiagonalMovement                       | Der Turm sollte sich nicht diagonal bewegen können. (keine diagonalen Felder in der validMoves Liste)                                                                                 |
| FileLeaderboardRepoTest#testMultiplePlayersAndSorting | Wenn mehrere Spieler im Leaderboard stehen, sollten alle Spieler, nach Anzahl der Gewinne absteigend sortiert, ausgegeben werden.                                                     |
| FileGameStateRepoTest#testLoadGameStateByFileName     | Wenn ein GameState Objekt gespeichert wurde, soll man es mit dessen Filenamen wieder laden können und dann sollten die Attribute mit dem Objekt das gespeichert wurde übereinstimmen. |

## ATRIP: Automatic

*[Begründung/Erläuterung, wie 'Automatic' realisiert wurde]*

Um alle Tests auszuführen, kann mittels eines Knopfdrucks (wenn es eine solche Run Configuration gibt und diese bereits ausgewählt ist) das Testen angestoßen werden.

Alle Testklassen besitzen setUp() Methoden die z. B. das Board vorbereiten mit dem Test ausgeführt wird. Bei den Testklassen im Persistence Package werden alle bestehenden Ordner und Testfiles vor und nach der Ausführung gelöscht.

Alle Tests besitzen assert Statements (mindestens 1) an denen abgelesen werden kann was nicht geklappt hat wenn Tests fehlschlagen.

## ATRIP: Thorough

*[jeweils 1 positives und negatives Beispiel zu 'Thorough'; jeweils Code-Beispiel, Analyse und Begründung, was professionell/nicht professionell ist]*

### Positiv:

FileGameStateRepoTest#testLoadGameStateByFileName Lines 72-84:

```
@Test
void testLoadGameStateByFileName() {
    GameState state = new GameState("white", "dummyBoard", "dummyHistory", "first",
    "second");
    repository.saveGameState(state);
    List<String> names = repository.listGameStateFileNames();
    assertFalse(names.isEmpty(), "Es sollte mindestens ein gespeicherter Spielstand
    vorhanden sein.");
    String fileName = names.getFirst();
    Optional<GameState> loaded = repository.loadGameState(fileName);
    assertTrue(loaded.isPresent(), "Der Spielstand sollte anhand des Dateinamens geladen
    werden können.");
    GameState loadedState = loaded.get();
    assertEquals(state.activePlayer(), loadedState.activePlayer(), "Der aktive Spieler muss
    übereinstimmen.");
    assertEquals(state.boardState(), loadedState.boardState(), "Der Board-Status muss
    übereinstimmen.");
    assertEquals(state.moveHistory(), loadedState.moveHistory(), "Die MoveHistory muss
    übereinstimmen.");
}
```

Dieser Test deckt mehrere Fälle ab:

- wenn ein GameState gespeichert wurde sollte auch mind. Einer vorhanden sein.
- Ein GameState sollte anhand seines Filenamens geladen werden
- Verschiedene Attribute (aktiver Spieler, Spielbrettaufbau, MoveHistory) von GameState der gespeichert wurde müssen mit dem geladenen GameState übereinstimmen.

Durch die Vielzahl getesteter Szenarien gilt der Test als „thorough“, da er alle wichtigen Aspekte der Zuglogik abdeckt. Er ist so gestaltet, dass spätere Änderungen an der Bewegungslogik sofort zu Fehlern führen, wenn sie nicht alle Fälle berücksichtige

### Negativ:

PawnTest#testInvalidBackwardMove() Lines 65-68:

```
void testInvalidBackwardMove() {
    board.printBoard();
    assertFalse(whitePawn.getValidMoves(board).contains(new Position(0, 4)), "Bauer darf
    nicht rückwärts ziehen.");
}
```

Dieser Test prüft lediglich ob sich ein Bauer nicht rückwärts bewegen kann und testet nicht auf Seitwärts-/ oder illegale Diagonalbewegungen.

Weil nur ein einziger Fall getestet wird, ist der Test nicht „thorough“. Er deckt nicht alle Aspekte der Bewegungslogik ab.

## ATRIP: Professional

*[jeweils 1 positives und negatives Beispiel zu 'Professional'; jeweils Code-Beispiel, Analyse und Begründung, was professionell/nicht professionell ist]*

### zwei Positiv Beispiele | 1.:

Der bei **ATRIP: Thorough** gezeigt Test wird hier auch als positiv Beispiel betrachtet. Er deckt alle relevanten Datenfelder ab und verwendet klare, aussagekräftige Assertion-Fehlermeldungen.

Dieser Test ist „professional“, da er strukturiert, wartbar und selbsterklärend ist. Er minimiert „Magic Numbers“ und unspezifische Fehlermeldungen.

### 2.:

```
@Test
void testEnPassant() {
    Pawn whitePawn = new Pawn("white", new Position(4,3));
    board.placePiece(whitePawn, new Position(4,3));
    board.printBoard();

    boolean moveOk = board.movePiece(new Position(6,4), new Position(4,4), "black");
    //schwarzer Pawn zieht 2 Felder
    assertTrue(moveOk, "Der schwarzen Bauer sollte zwei Felder gezogen sein.");

    List<Position> validMoves = whitePawn.getValidMoves(board);
    assertTrue(validMoves.contains(new Position(5,4)),
        "Der weiße Bauer sollte f5 (en passant) als gültigen Zug haben.");
}
```

Dieser Test deckt alle relevanten Datenfelder ab und verwendet klare, aussagekräftige Assertion-Fehlermeldungen.

Dieser Test ist „professional“, da er strukturiert, wartbar und selbsterklärend ist. Er hat außerdem aussagekräftige Fehlermeldungen.

## Code Coverage

*[Code Coverage im Projekt analysieren und begründen]*

Die folgenden Bilder zeigen die Code Coverage. Das erste Bild zeigt die Packages und die nachfolgenden Bilder zeigen die Klassen in den jeweiligen Packages.

### Coverage Breakdown

| Package ▲           | Class, %   | Method, %     | Branch, %       | Line, %         |
|---------------------|------------|---------------|-----------------|-----------------|
| Application         | 20% (1/5)  | 7,1% (1/14)   | 0% (0/32)       | 0,8% (1/121)    |
| Application.Command | 0% (0/2)   | 0% (0/2)      |                 | 0% (0/2)        |
| Domain.Board        | 100% (2/2) | 89,3% (25/28) | 64,7% (88/136)  | 62,1% (121/195) |
| Domain.Exceptions   | 0% (0/2)   | 0% (0/2)      |                 | 0% (0/2)        |
| Domain.Game         | 80% (4/5)  | 50% (10/20)   | 5,9% (2/34)     | 24,5% (23/94)   |
| Domain.Pieces       | 100% (7/7) | 100% (29/29)  | 91,4% (117/128) | 100% (139/139)  |
| Domain.Player       | 0% (0/2)   | 0% (0/4)      |                 | 0% (0/6)        |
| Persistence         | 100% (3/3) | 95,7% (22/23) | 51,9% (28/54)   | 79% (113/143)   |

## Package Application:

| Class ▲                          | Class, %   | Method, % | Branch, % | Line, %   |
|----------------------------------|------------|-----------|-----------|-----------|
| <a href="#">CLIController</a>    | 0% (0/1)   | 0% (0/5)  | 0% (0/14) | 0% (0/30) |
| <a href="#">GameInitializer</a>  | 0% (0/1)   | 0% (0/4)  | 0% (0/18) | 0% (0/73) |
| <a href="#">GameStateWrapper</a> | 0% (0/1)   | 0% (0/1)  |           | 0% (0/1)  |
| <a href="#">LeaderboardEntry</a> | 100% (1/1) | 50% (1/2) |           | 50% (1/2) |
| <a href="#">QuickStart</a>       | 0% (0/1)   | 0% (0/2)  |           | 0% (0/15) |

## Package Domain.Board:

| Class ▲                  | Class, %   | Method, %     | Branch, %      | Line, %         |
|--------------------------|------------|---------------|----------------|-----------------|
| <a href="#">Board</a>    | 100% (1/1) | 86,4% (19/22) | 63,5% (66/104) | 59,9% (103/172) |
| <a href="#">Position</a> | 100% (1/1) | 100% (6/6)    | 68,8% (22/32)  | 78,3% (18/23)   |

## Package Domain.Game:

| Class ▲                     | Class, %   | Method, %  | Branch, % | Line, %       |
|-----------------------------|------------|------------|-----------|---------------|
| <a href="#">Game</a>        | 0% (0/1)   | 0% (0/6)   | 0% (0/24) | 0% (0/54)     |
| <a href="#">GameRecord</a>  | 100% (1/1) | 50% (1/2)  |           | 33,3% (1/3)   |
| <a href="#">GameState</a>   | 100% (1/1) | 100% (3/3) | 50% (1/2) | 83,3% (10/12) |
| <a href="#">Move</a>        | 100% (1/1) | 100% (3/3) | 50% (1/2) | 88,9% (8/9)   |
| <a href="#">MoveHistory</a> | 100% (1/1) | 50% (3/6)  | 0% (0/6)  | 25% (4/16)    |

## Package Domain.Pieces

| Class ▲                            | Class, %   | Method, %  | Branch, %     | Line, %      |
|------------------------------------|------------|------------|---------------|--------------|
| <a href="#">AbstractChessPiece</a> | 100% (1/1) | 100% (4/4) |               | 100% (6/6)   |
| <a href="#">Bishop</a>             | 100% (1/1) | 100% (4/4) | 100% (16/16)  | 100% (22/22) |
| <a href="#">King</a>               | 100% (1/1) | 100% (4/4) | 100% (16/16)  | 100% (17/17) |
| <a href="#">Knight</a>             | 100% (1/1) | 100% (4/4) | 65% (13/20)   | 100% (17/17) |
| <a href="#">Pawn</a>               | 100% (1/1) | 100% (5/5) | 92,1% (35/38) | 100% (35/35) |
| <a href="#">Queen</a>              | 100% (1/1) | 100% (4/4) | 95% (19/20)   | 100% (22/22) |
| <a href="#">Rook</a>               | 100% (1/1) | 100% (4/4) | 100% (18/18)  | 100% (20/20) |

## Package Persistence:

| Class ▲                             | Class, %   | Method, %   | Branch, %    | Line, %       |
|-------------------------------------|------------|-------------|--------------|---------------|
| <a href="#">FileGameRecordRepo</a>  | 100% (1/1) | 100% (8/8)  | 50% (15/30)  | 86,4% (57/66) |
| <a href="#">FileGameStateRepo</a>   | 100% (1/1) | 100% (8/8)  | 50% (6/12)   | 71,1% (27/38) |
| <a href="#">FileLeaderboardRepo</a> | 100% (1/1) | 85,7% (6/7) | 58,3% (7/12) | 74,4% (29/39) |



Bei trivialen Klassen (Application.Player), leeren Interfaces (Application.Command) und Exceptions (Domain.Exceptions) ist der Testaufwand im Verhältnis zum Nutzen nicht gerechtfertigt. Da sie weder komplexe Logik noch Verarbeitungsalgorithmen enthalten, entsteht wenig Mehrwert, wenn sie isoliert getestet würden.

Die Game Klasse enthält die zentrale Spielschleife, die während des Spielablaufs dauerhaft läuft. Diese Komponente ist häufig stark mit der Benutzeroberfläche und Echtzeiteingaben gekoppelt.

Eine vollständige Automatisierung der Spielschleife würde ein aufwändiges Simulieren von Benutzerinteraktionen und Zustandsänderungen erfordern, was den Nutzen von Unit Tests in diesem Fall übersteigt.

Anstatt die Spielschleife isoliert zu testen, wurde der Schwerpunkt auf Unit Tests für die zentralen Spielmechanismen (wie Zugvalidierung, Schach-/Matt-Erkennung usw.) gelegt. Diese Komponenten sind leichter zu isolieren und gewährleisten die korrekte Funktionalität der Kernlogik

## Fakes und Mocks

*[Analyse und Begründung des Einsatzes von 2 Fake/Mock-Objekten; zusätzlich jeweils UML Diagramm der Klasse]*

Dieses Projekt verwendet keine Fake/Mock-Objekte. Es werden dummy Strings für die Persistenztests verwendet („dummyBoard“ und „dummyHistory“).

```
GameState state = new GameState("white", "dummyBoard", "dummyHistory", "first", "second");
```

## Kapitel 6: Domain Driven Design

### Ubiquitous Language

*[4 Beispiele für die Ubiquitous Language; jeweils Bezeichnung, Bedeutung und kurze Begründung, warum es zur Ubiquitous Language gehört]*

| Bezeichnung            | Bedeutung                                                                                                                                    | Begründung                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zug (Move)             | Einzelner Spielzug, eine Schachfigur von einer Startposition zu einer Zielposition bewegen                                                   | alle Spielregeln, Validierungen (z. B. ob ein Zug legal ist) und die Spielprotokollierung drehen sich um die Erfassung und Bewertung von Zügen.                       |
| Schachmatt (Checkmate) | Spielsituation, in der der König im Schach steht und keinen legalen Zug hat → Spielende                                                      | definiert den Endzustand eines Spiels, wird von Spielern und in der Domänenspezifikation genutzt.                                                                     |
| En Passant             | Sonderzug, ein Bauer kann einen gegnerischen Bauern schlagen, der im vorherigen Zug einen Doppelzug gemacht hat und an ihn vorbeigezogen ist | ist ein spezifischer Begriff im Schach, der eine besondere Regel beschreibt. Ist in der Domäne fest verankert ist und erfordert spezielle Logik in der Zugberechnung. |
| Promotion              | Ein Bauer erreicht die                                                                                                                       | gegnerischeverändert das Material und die Dynamik einer                                                                                                               |



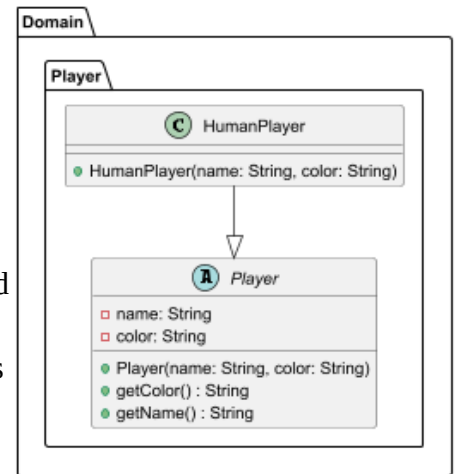
Grundreihe und wird er in eine andere Figur (hier immer Dame) umgewandelt.

Stellung. Alle Schachspieler, Domänenexperten und Entwickler wissen genau, was unter Promotion zu verstehen ist.

## Entities

*[UML, Beschreibung und Begründung des Einsatzes einer Entity; falls keine Entity vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]*

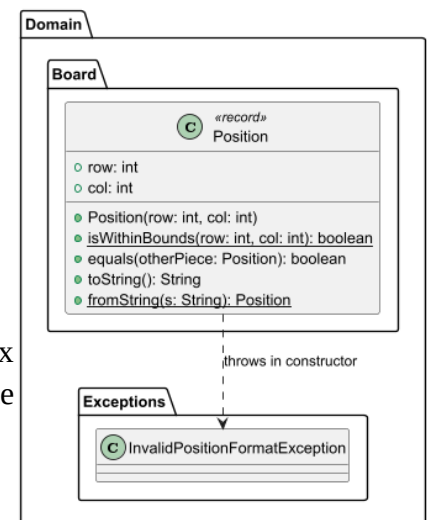
**Player:** Ein Spieler hat eine eindeutige Identität (z. B. Name, Farbe) und seine Identität bleibt über den gesamten Spielverlauf hinweg konstant. Da ein Spieler über das Spiel hinweg eindeutig identifizierbar sein muss (und beispielsweise auch persistiert oder in der Bestenliste berücksichtigt wird), ist es sinnvoll, ihn als Entity zu modellieren.



## Value Objects

*[UML, Beschreibung und Begründung des Einsatzes eines Value Objects; falls kein Value Object vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]*

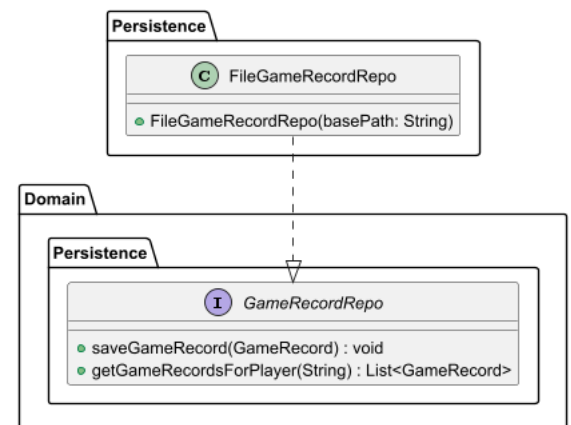
**Position:** Eine Brettcoordinate, die durch den Zeilen- und Spaltenindex bestimmt wird (z. B. "e2"). Eine Position ist rein durch ihre Werte definiert, ist unveränderlich und hat keine eigene Identität.



## Repositories

*[UML, Beschreibung und Begründung des Einsatzes eines Repositories; falls kein Repository vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]*

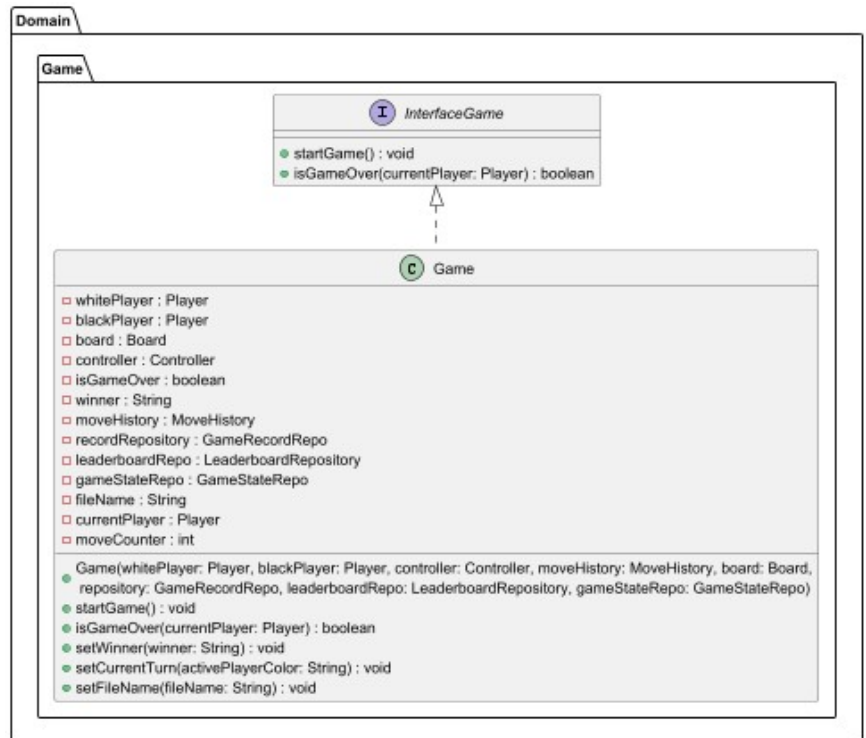
**GameRecordRepo:** Schnittstelle zum Speichern und Abrufen abgeschlossener Spielaufzeichnungen (GameRecords). Kapselt die Persistenzlogik und ermöglicht der Domain-Schicht, sich auf abstrakte Operationen zu verlassen, ohne von konkreten I/O-Details abhängig zu sein.



## Aggregates

*[UML, Beschreibung und Begründung des Einsatzes eines Aggregates; falls kein Aggregate vorhanden: ausführliche Begründung, warum es keines geben kann/hier nicht sinnvoll ist]*

**Game:** Das Game-Objekt kapselt alle relevanten Komponenten eines Schachspiels (Board-Zustand, MoveHistory und beteiligten Spieler). Als Aggregate Root steuert Game den gesamten Zustand einer Partie. Alle Änderungen an Figuren, Zügen und dem Spielverlauf erfolgen über das Game-Objekt, wodurch die Konsistenz der gesamten Spiellogik gewährleistet ist.



## Kapitel 7: Refactoring

### Code Smells

*[jeweils 1 Code-Beispiel zu 2 Code Smells aus der Vorlesung; jeweils Code-Beispiel und einen möglichen Lösungsweg bzw. den genommen Lösungsweg beschreiben (inkl. (Pseudo-)Code)]*

#### Duplicated Code:

Board.java Lines 140-142:

```
public boolean isWithinBoard(Position newPos) {
    return newPos.row() >= 0 && newPos.row() <= 7 && newPos.col() >= 0 && newPos.col() <= 7;
}
```

Position.java Lines 13-15:

```
public static boolean isWithinBounds(int row, int col) {
    return row >= 0 && row <= 7 && col >= 0 && col <= 7;
}
```

Lösungsweg ab Commit 9e88e16

(<https://github.com/culmets/cutesRepo/commit/9e88e1663d81343bad4e51e2fad17a7dc29146a1>):

```
public boolean isWithinBoard(Position newPos) {
    return Position.isWithinBounds(newPos.row(), newPos.col());
}
```

## Long Method:

## 2 Refactorings

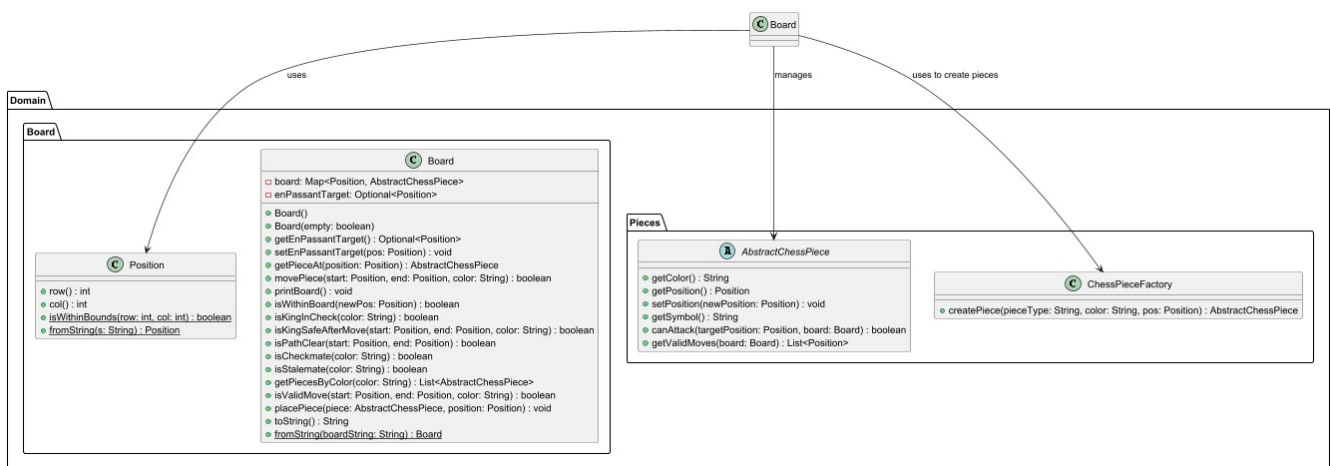
*[2 unterschiedliche Refactorings aus der Vorlesung anwenden, begründen, sowie UML vorher/nachher liefern; jeweils auf die Commits verweisen]*

**Replace conditional with polymorphism (Commits 6170639/577f46e):** siehe S. 7 Kapitel 3 - SRP für den Code

Mit der Factory wird der Code so strukturiert, dass er für Erweiterungen offen ist (z. B. das Hinzufügen neuer Schachfiguren), aber für Änderungen an bestehenden Klassen geschlossen bleibt. Das switch-Statement wird durch eine Map ersetzt. Das macht den Code übersichtlicher und leichter wartbar macht. Die Entscheidung, welche konkrete Schachfigur erstellt wird, ist klar von der restlichen Logik getrennt.

Durch das Auslagern der Erstellung der Schachfiguren in eine Factory-Klasse erhält jede Klasse eine klar definierte Verantwortung. Die Methode fromString() muss sich nicht mehr um die Details der Objekterstellung kümmern.

Vorher-UML vom Board siehe S. 5



**Extract Method: getValidMoves() in allen ChessPieces, hier wird der Pawn betrachtet**

**Commit 975c1a6**

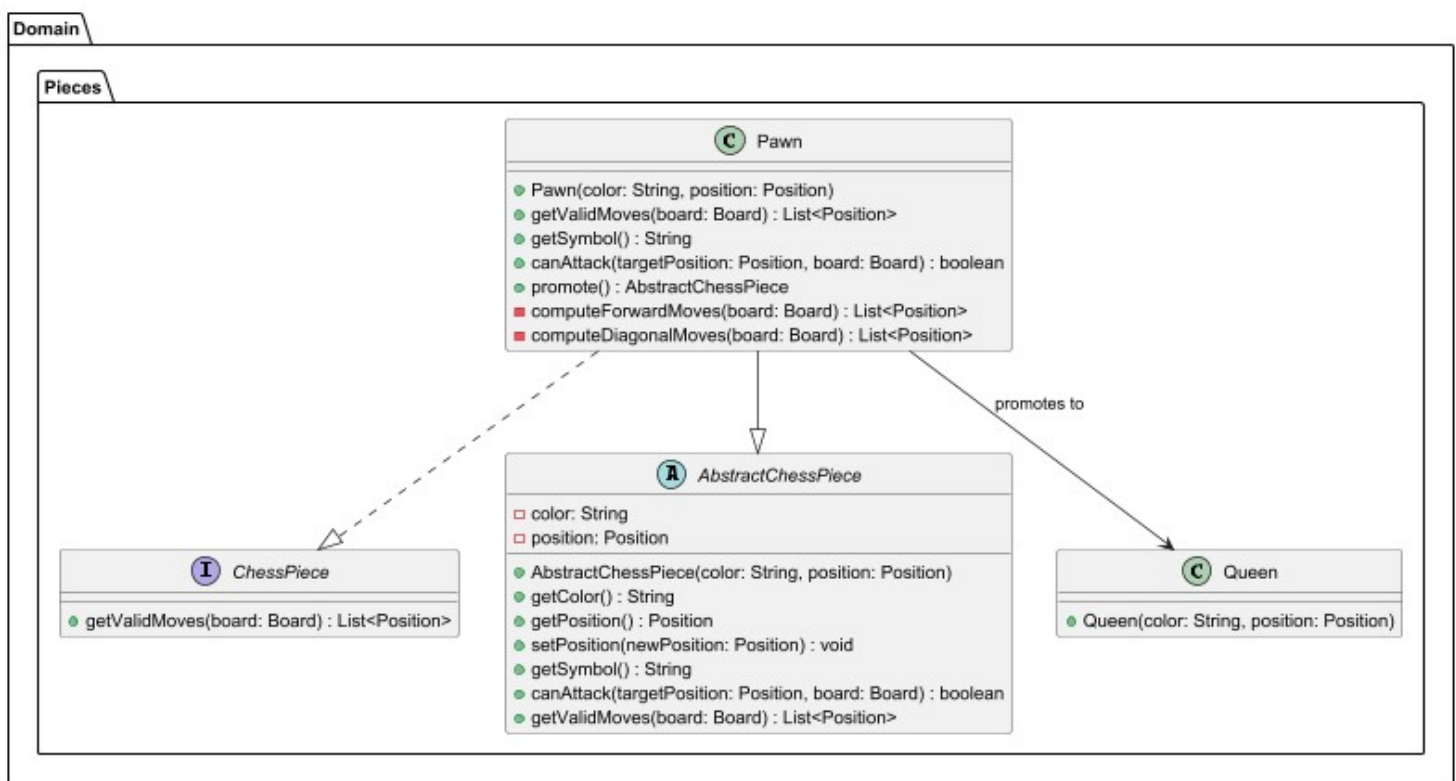
Die große getValidMoves() Methode wurde in mehrere Sub-Methoden aufgeteilt.

**computeForwardMoves():** Diese Methode prüft, ob das Feld direkt vor dem Bauern frei ist und ob vom Startfeld aus ein Doppelzug möglich ist.

**computeDiagonalMoves(Board board):** Diese Methode berechnet alle diagonalen Züge. Sowohl die normalen Schlagzüge (wenn eine gegnerische Figur vorhanden ist) als auch den En Passant-Zug, falls das entsprechende Zielfeld als En Passant-Ziel gesetzt wurde.

Durch die Aufteilung der Logik in zwei separate Methoden wird jede Methode kleiner und fokussierter. Änderungen oder Erweiterungen (z. B. zusätzliche Regeln für En Passant) können so isoliert vorgenommen und separat getestet werden.

Vorher-UML für Pawn siehe S.7. Hier das nacher-UML:



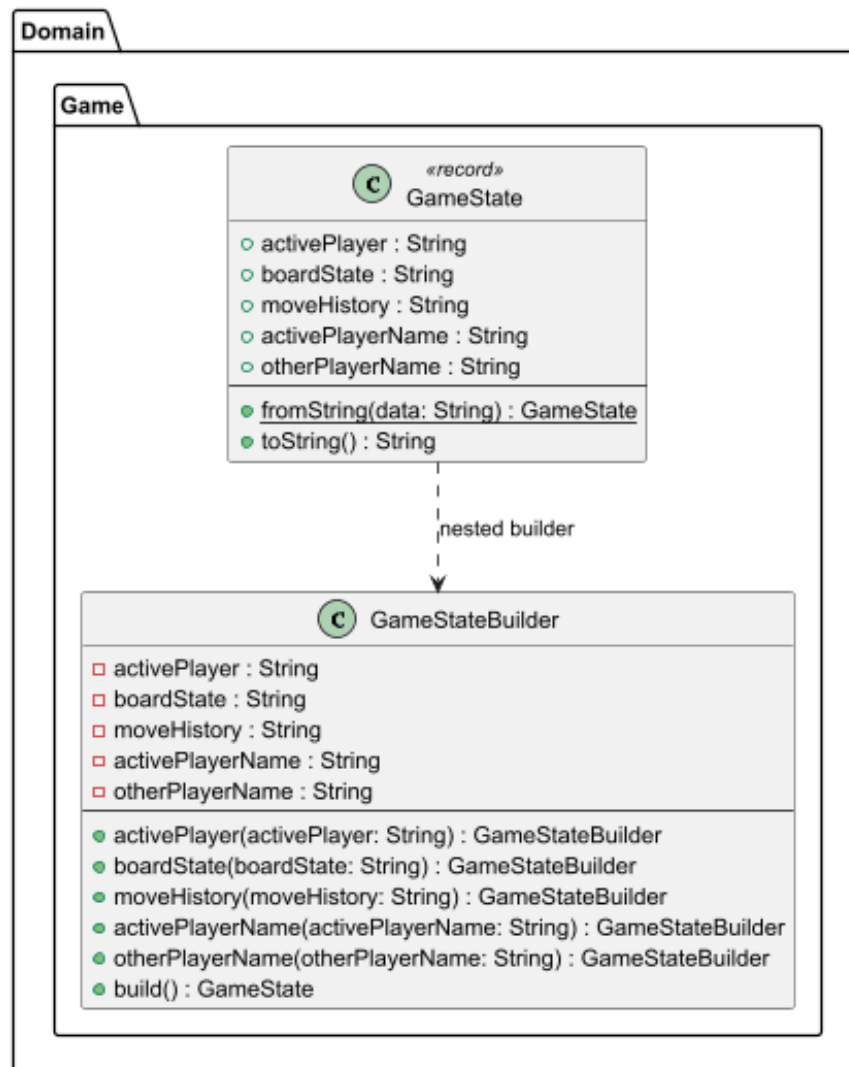
## Kapitel 8: Entwurfsmuster

*[2 unterschiedliche Entwurfsmuster aus der Vorlesung (oder nach Absprache auch andere) jeweils sinnvoll einsetzen, begründen und UML-Diagramm]*

### Entwurfsmuster: Builder Pattern

Das Builder Pattern wurde für die Erstellung von GameState-Objekten implementiert, um die Konstruktion komplexer Spielzustände übersichtlich und flexibel zu gestalten. Statt eines schwer lesbar und wartbaren Konstruktor mit vielen Parametern zu verwenden, werden mit dem Builder schrittweise

aller relevanten Felder (Spielernamen, Board-Zustand, MoveHistory) zusammengestellt. Dadurch wird die Codequalität verbessert, weil Änderungen an der Struktur des GameState nur an einer zentralen Stelle vorgenommen werden müssen.



## Entwurfsmuster: Factory Pattern

Beim einlesen eines BoardStates um es von einem String zu einem Board (Objekt) zu machen wird eine `ChessPieceFactory` verwendet. Dadurch wird die Erstellung von Objekten von der Geschäftslogik getrennt. Die Kernkomponenten (Domain) werden von technischen Details isoliert. Die Factory kapselt den Erstellungsprozess und stellt sicher, dass die Domain-Klassen nicht direkt von konkreten Implementierungen abhängig sind.

Die Factory sorgt dafür, dass hochrangige Module (z. B. Spiellogik) nicht von niederrangigen, konkreten Klassen abhängen. Abhängigkeiten werden über abstrakte Schnittstellen oder Factory-Methoden injiziert. Konkrete Implementierungen können bei Bedarf einfach ausgetauscht werden.

Mit der Factory können neue Schachfiguren oder Änderungen in der Objekterzeugung vorgenommen werden, ohne dass bestehende Geschäftslogik angepasst werden muss. Das bestärkt außerdem das Open-Closed-Prinzip, welches besagt, dass Module offen für Erweiterungen, aber geschlossen für Modifikationen sein sollten.

